# Initiation à Python

M. L-Lahlou

UNIVERSITE CADI AYYAD
Faculté des Sciences Semlalia Marrakech
Département de Mathématiques

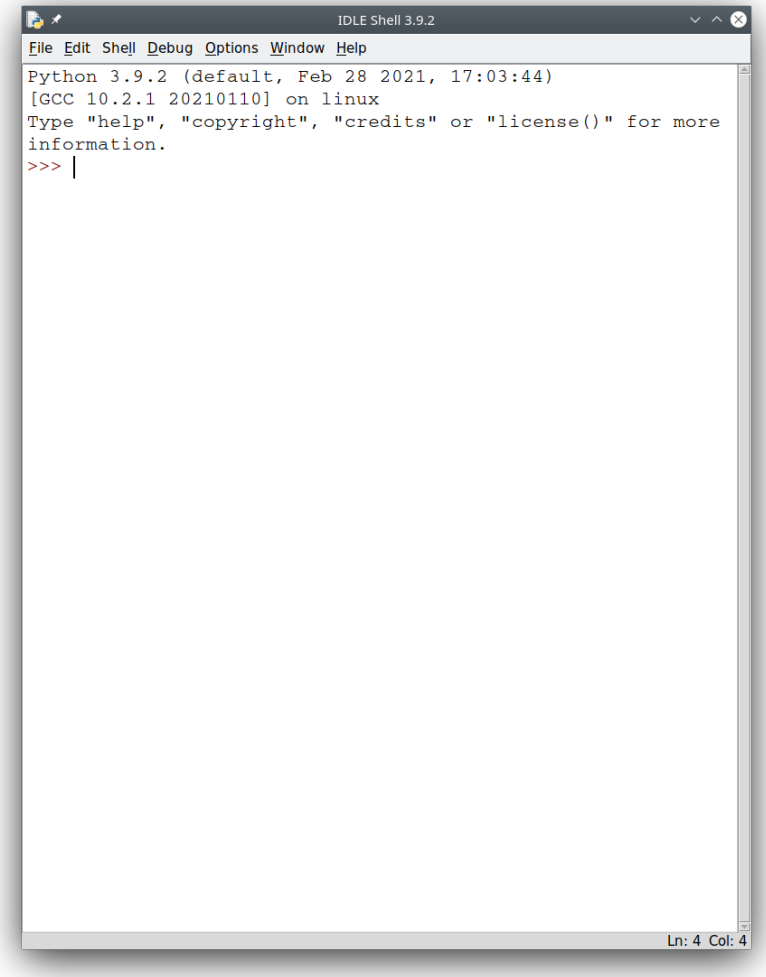
**SMA4/SMI4, Printemps 2023**
**Analyse Numérique**
**Travaux Pratiques**

## 1 Introduction

- **Python** est un langage de programmation libre multi-usage : outils système, développement web, multimedia, calcul scientifique ... etc.
- Python et tout son environnement scientifique pour **PC** est diffusé dans une distribution nommée **Anaconda**, téléchargeable à partir de l'adresse web :
  https://www.anaconda.com/products/distribution.
- Attention : choisir en bas de la page la plate-forme (Windows, Linux ou MacOs) et le processeur de la machine (32bits ou 64 bits).
- Il y a plusieurs environnements pour le développement en python. On emploie au choix les interfaces **idle** ou **spyder**. Un programme en python est appelé **script**.
- Il existe plusieurs applications pour coder en python sur **smartphones android**. On conseille par exemple d'installer **pydroid**.

## 2 Démarrage

- Pour **idle** sous Linux on tape la commande `idle` en console ou à partir du menu des applications.
  Sous Windows, on lance du menu `Anaconda Prompt`, qui charge une console, puis taper `idle`.
  **On conseille idle pour les ordinateurs à faibles ressources** :




- Pour **spyder** sous Linux on tape la commande `spyder` en console ou à partir du menu des applications. Sous Windows, on lance directement du menu `spyder`.

Cette fenêtre est **le shell** appelée aussi **console** de python. Il suffit de taper une instruction puis d'appuyer sur la touche « Entrée » de votre clavier pour lancer son exécution. C'est le *mode interactif*.
Travaillez en mode intractif de la section 3 à la section 6.

# 3 Nombres, expressions numériques et chaînes

## 3.1 Expressions numériques

Dans python tous les calculs sont numériques. Les nombres sont soit

- des entiers comme 3, -12, ...
- soit en virgule flottante : 1.53, -4.0, 2.7e-6, ... en double précision. Remarquer pour saisir en notation scientifique $10^{-9}$, on fait `1e-9 (ou 1E-9)`.

Les opérateurs arithmétiques usuels sont :

| | |
|---|---|
| `+,-` | addition, soustraction |
| `*,/` | multiplication, division décimale |
| `**` | puissance `x**y` : $x^y$ |
| `//` | division euclidienne |
| `%` | reste de division euclidienne |

- Une chaine est du texte encadré par des guillemets (") ou des apostrophes ('). Exemples `"python est un langage"`, `'24'`

convertir un nombre en chaine et inversement :

- `str()` : convertit un nombre en chaine. Par exemple `str(61)` convertit en chaine `'61'`
- `int()` ou `float()` : convertit une chaine en nombre entier ou flottant. Par exemple `float("-4.125")` convertit en nombre -4.125

```
print(6+(3.5-1)/2+3**2) #print pour afficher
```

- Les priorités des opérateurs sont comme dans le langage C. Mais puissance ** est prioritaire sur tous les autres.
- On observe que tout ce qui est saisi après le caractère # sur la même ligne est un commentaire.

*Remarque* : Si vous vous trompez dans une instruction, il suffit de cliquer dessus, le shell la reédite au niveau de l'invite " >>> " et vous corrigez.

quelques opérations sur les chaines :

```
print("hello"+' '+"world") #cancatenation
print(4*"world") #dupliquer une chaine
```

```
ch="je programme en python"
print(ch[3:7]) #extraire une partie de chaine
```

### 3.2 Expressions logiques

- Un objet booléen a deux valaurs `True` (Vrai) et `False` (Faux).
- Toute expression de valeur numérique non nulle est considérée comme vraie.
- ***Expressions booleenes :*** sont formées par

| Opération | Notation |
|---|---|
| Comparaison de valeurs | `==,!=,<,<=,>,>=` |
| Logique | `and,or,not()` |
| Identité | `is, is not` |
| Appartenance | `in, not in` |

```
1<=2 and -4>5 # faux
```

```
not(1>10) # vraie
```

```
'h'  in "blabla"# faux
```

# 4 Variables

Toute variable doit être initialisée ou affectée.

```
print(a) # erreur!!
```

```
a=3
b=a/2
print(a)
print(b)
```

# 5 Lire et Afficher

- **Affichage simple**

```
print(objet) #  affiche objet

#Pour afficher plusiers objets séparés par des blancs:
print(objet1,objet2, ...)
```

```
x,y=2,4.5 # affectation simultanée de variables
print(x,y)
z=x+y
print("La somme vaut : ",z)
```

- **Affichage formaté**
  On peut afficher des chaines qui contient du texte et des données en utilisant les codes de format selon la syntaxe :
  `print(" ...  {:code1} ...  {:code2} ...  ".format(var1,var2,...))`

```
a=1/8
print("a= {:f}".format(a)) #decimal libre
print("a= {:.4f}".format(a)) #decimal 4 chiffres apres la virgule
print("a = {:.5e}".format(a)) #decimal notation scientifique

b=2**5.25
print("a= {:.4f}, b= {:.3E}".format(a,b))
```

Comme dans langage C, `print()` supporte les caractères spéciaux : `\n`,`\t`, ...

- **Lecture** : `input(" ... ")`

```
ch=input("tapez votre nom")
"""
affiche le message et affecte
ce qui est saisi au clavier
à une variable chaine
"""

ch #ch est une chaine
print(ch)
```

*Remarque* : un texte encadré par triple guillemets est un commentaire en plusieurs lignes

```
n=input("donner un nombre entier : ")
print(n+3) # produit une erreur!
```

Ce qu'on introduit au clavier après l'exécution de `input(" ... ")` est toujours une chaine.
Si on veut saisir et stocker en nombre, il faut convertir le résultat de la fonction `input()` à un type numérique à l'aide des fonctions `int()` ou `float()`.

```
#par exemple :
n=int(input("donner un nombre entier : "))
print(n+3)
```

Plus généralement `eval(input(" ... "))` si on veut lire à partir du clavier plusieurs données en une seule ligne ou même des objets composés comme les listes (voir plus loin) :

```
x,y=eval(input("saisir respectivement 2 nombres : "))
print(x)
print(y)
# on introduit 2 nombres au clavier séparés par une virgule affectés
# dans l'ordre à x et y
```

## 6 Listes

une liste est une collection d'objets dont la syntaxe est : `[objet1,objet2, ...]`

```
L=[3,-1,'bonjour',6]
```

- On accède à chaque élément par sa position.
- Les positions commencent toujours par 0
- La fonction **len** donne la taille de la liste

```
print(L[0])
print(L[2])
print(len(L)) # taille
```

Quelques méthodes opérant sur les listes :

```
l=[3,5]
l.append(-8) # ajouter un élément
```

```
print(l)
```

```
L1=[5,'hello']
print(l+L1) #fusionner
```

```
Ls=3*L1 #duplique une liste
print(Ls)
```

```
Ls2=[2,4,'f',12,-5,3,-4]
print(Ls2[2:5]) #extraire une partie de la liste
```

---

**Nous travaillons à partir de maintenant dans la fenêtre d'édition**.
Pour editer des programmes python appelés **scripts**, on fait `File->New File (ou Ctrl N)`. Une seconde fenêtre s'ouvre dans laquelle on rédige du code. Les actions suivantes sont utiles :

- Ouvrir un programme existant : `File->Open Module (ou Ctrl M)`
- Enregistrer un programme : `File-> Save (ou Ctrl S)` : **doit avoir l'extension .py**
- Executer un programme : menu `Run -> Run Module (ou F5)`. Les résultats sont affichés dans la fenêtre du shell.

  **On peut interrompre toute exécution qui plante pour une queconque raison : dans le shell, aller au menu `Shell-Interrupt Execution` ou `Ctrl C`.**

Créez un fichier `TP0.py` contenant les instructions du reste de l'initiation.
*Pour ne pas charger le fichier, effacez chaque manipulation ou bien utilisez les commentaires pour passer à la suivante.*

## 7 Instructions de contrôle

Une instruction de contrôle est formée par une **en-tête** et un **bloc**.

Chaque bloc a un niveau de profondeur ou **indentation**.

```
en-tete: #attention aux 2 points au bout de en-tete
    instruction 1
    instruction 2
    ...
    ... #instructions indentées
#Fin bloc
...
...
autres_instructions
hors bloc alignes avec en-tete
```

## 7.1 Instructions alternatives

Les instructions alternatives ou branchements sont gérées par les mots

```
if condition:  # condition est une expression logique
    bloc_instructions
    # Indentation obligatoire

#ou bien

if condition:
    bloc_instructions
else: # alignement obligatoire avec if
    bloc_instructions


#plus généralement :

if condition_1:
    bloc_1
elif condition_2:
    bloc_2

    # ... ainsi de suite

else:
    bloc
```

```
#Exemple :
x,y=eval(input("donner 2 nombres : "))
if x-y>0:
    print("x est plus grand que y") # indentation
elif x==y: # aligne avec if
    print(" x et y sont egaux")
else:
    print("x est plus petit que y")
```

## 7.2 Boucles

Python possède 2 types de boucles :

### 7.2.1 Boucle for : boucle bornée

La boucle **for** est adaptée pour parcourir n'importe quelle séquence (par ex. une liste ou une chaîne), dans l'ordre où ils apparaissent dans la séquence.

```
for objet in sequence:
    instruction 1
    instruction 2
```

```
for x in [5,2.5,-7,[-1,3]]:
    print(x)
```

La fonction **range** : très utile pour les boucles `for`.

- `range(debut,fin,pas)` construit une séquence d'entiers entre **debut** et **fin** avec un avancement **pas**.
- la borne **fin** toujours exclue.
- **pas** peut être negatif.

```
for i in range(1,15,2):
    print('i= ',i,'=> son carre = ',i**2)
```

```
for j in range(10,1,-1):
    print('j= ',j,'=> son inverse = ',1/j)
```

### 7.2.2 Boucle while : boucle non bornée

La boucle **while** est une itération plus générale. Elle exécute le bloc indenté tant que la condition de test est réalisée.

```
while condition:
    instruction 1
    instruction 2
    ...
```

```
a=10
while a>0.1:
    a=a/5
    print(a)
```

instructions `break` et `continue`

```
for i in range(0,10,1):
    if i==6:
        break # interrompre la boucle
    x=i+1
    print(x)
```

```
i=0
while i<9:
    i=i+1
    if i==3:
        continue
    print(i)
```

# 8 Fonctions

- On peut définir de nouvelles fonctions à l'aide du modèle :

```
def nomFonction(arg1,arg2,...):
    instruction1
    instruction2
    ...
    return(res1,res2,...) # return non obligatoire,
                          #obligatoire si la fonction renvoie des résultats.
# fin bloc
```

où arg1,arg2,... sont les arguments d'entrée de la fonction et res1,res2,... sont les résultats.

- On appelle une fonction par `nomFonction(param1,param2,...)`, où `param1,param2,...` sont les paramètres effectifs dans l'ordre de définition des arguments.
- Une fonction peut ne pas avoir d'argument, ou admettre tout type d'arguments : nombres, chaines, listes, fonctions, ...
- Elle peut retourner un résultat, plusieurs résultats, ou ne pas retourner de résultat
- Elle peut retourner tout type de résultat.

```
def som(a,b):
    s=a+b
    return(s)
```

```
#appel de fonction

print("somme = ",som(7,3))

a,b=-0.5,3.25
c=som(a,b)
print("somme = ",c)
```

- Fonction à plusieurs résultats :

```
def somDiff(a,b):
    s=a+b
    d=a-b
    return(s,d)
```

```
#appel
print(somDiff(3,5))

#ou bien
x,y=somDiff(3,5) #affecter resultats a deux variables
print("x= ",x,",y= ",y)
```

- fonction en argument :

```
def MyFunc(x,fonc):
    y=-1+fonc(x)
    return(y)
```

```
def g(u):
    return(u**3)

x=2
z=MyFunc(x,g)
print("resultat de MyFunc = ",z)

def h(t):
    return(1/t)
print("Autre resultat de MyFunc = ",MyFunc(4,h))
```

**Fonctions mathématiques usuelles:**
Pour pouvoir utiliser les mathématiques, il faut importer d'abord dans le script la bibliothèque standard des mathématiques à l'aide de la ligne de code `from math import *`.
La notation python de quelques constantes et fonctions mathématiques usuelles sont dans la table qui suit :

| | |
|---|---|
| `pi` | constante $\pi$ |
| `e` | constante exp(1) |
| `abs` | valeur absolue |
| `exp` | exponentielle |
| `log (log10)` | logarithme népérien (en base 10) |
| `cos,sin,tan` | fonctions trigonométriques |
| `acos,asin,atan` | fonctions trigonométriques inverses |
| `cosh,sinh,tanh` | fonctions hyperboliques |
| `acosh,asinh,atanh` | fonctions hyperboliques inverses |
| `sqrt` | racine carrée |
| `floor` | partie entière |

```
from math import *
x=cos(pi)
print(x)
```